

Milestone 2 - Learned Compatibility for Jigsaw Reconstruction

CSE480 Machine Vision . Ain Shams University, Faculty of Engineering, Mechatronics Engineering Department . Summer 2026

1. Overview

Milestone 1 reconstructed a jigsaw from a hand-designed compatibility measure: shape continuity plus colour continuity, combined with fixed weights. This milestone replaces that measure with two **learned** ones - a Siamese convolutional network and a graph neural network - and compares all three on the same puzzles.

The comparison is deliberately narrow, and that is the point. Segmentation, contour tracing, corner location, side description and **the assembly algorithm itself** are all unchanged from Milestone 1. The only thing that differs between the three methods is the function that answers "*how well do these two sides fit?*". Everything downstream consumes the same `CompatibilityTable` interface, so any difference in the reconstructions is attributable to the matcher and not to a different pipeline.

```
segment -> describe pieces -> ??? classical: shape + colour formula ???  
                                     ??? Siamese CNN                               ??-> same assembler -> imag  
e + quality  
                                     ??? graph neural network                       ??
```

2. Task 1 - Dataset preparation

2.1 Where the labels come from

A training example is a **pair of sides** labelled 1 if they are genuinely adjacent in the finished puzzle and 0 otherwise. Producing that label requires knowing, for every side of every piece, which grid direction it faced before the pieces were shuffled.

The provided photographs cannot supply it. Their annotations give each piece's *identity*, which (as Milestone 1 §9.1 established) fixes which **pieces** are neighbours - but not which of a piece's four sides touches which, because the rotation a piece happens to be lying at in the photograph is unknown. Side-level labels are simply not recoverable from a bounding box and a class id.

Training therefore uses puzzles cut by the Milestone 1 generator, where the answer is known exactly by construction. Critically, every training sample is passed through **the same pipeline the classical matcher uses** - scattered onto a canvas, segmented by thresholding, traced, corner-located and described. The networks therefore see the same segmentation noise, the same corner-localisation error and the same colour sampling as the classical measure they are being compared against. Had they been fed the generator's exact geometry instead, any advantage they showed would have been an artefact of a cleaner front end.

A puzzle is discarded if segmentation does not recover every piece, or if the piece-to-ground-truth association is not one-to-one, so no unreliable label enters the set. A seam is also discarded if the two sides are not a tab/blank pair, since that indicates the description itself is wrong.

2.2 Positive and negative sampling

Positives are all the true seams of a puzzle. Negatives are drawn **only from admissible pairs** - a tab facing a blank on a different piece. This matters more than it looks: a model trained against arbitrary negatives would learn mostly to reject flat-to-flat and tab-to-tab pairings, score beautifully, and be worthless, because the classical matcher already excludes those for free with a hard constraint. The learned model must be better than chance *on the pairs that are genuinely in contention*.

Six negatives are drawn per positive, **half of them from the hardest available candidates** - the ones the classical measure ranks best - so the networks spend their capacity where the problem actually is rather than on easy rejections. Negatives are redrawn every epoch, so the models see far more of the negative space than a

single fixed draw would give.

2.3 Augmentation

Applied to the colour channels only; the shape channel is geometry and recolouring it would be meaningless. The four effects match those the brief lists:

Effect	Range	Why
Illumination	$\times 0.85$ -1.15	pieces lie under different light across a table
Contrast	$\times 0.8$ -1.25 about the mean	exposure varies between photographs
Colour balance	$\times 0.93$ -1.07 per channel	white balance drifts
Sensor noise	$\sigma = 0.02$ additive	camera noise

2.4 Splitting

Splits are by **puzzle, never by pair**. Two sides of one seam must not land on opposite sides of the split, and all pieces of one puzzle share a source picture, so a pair-level split would leak. 70 / 15 / 15 train / validation / test. The identical `SplitSpec` object is handed to both models, and the test split is not touched until the final evaluation.

The set actually used. 60 puzzles across three grid sizes (3x4, 4x5, 5x7), split 42 / 9 / 9 by puzzle. That yields **10,178 training pairs** (1,454 positive, 8,724 negative - a positive fraction of 0.143), with 2,100 for validation and 1,848 for test. Full record in `results/milestone2/dataset.json`.

3. Task 2 - Model development

The brief requires two *fundamentally different* models, and notes that changing layer counts, activations or hyper-parameters does not qualify. These two differ in their unit of inference, their input representation and their inductive bias.

3.1 Model 1 - Siamese convolutional network

Why this shape of model. A side's descriptor genuinely is a small picture: three colour channels sampled at eight depths just inside the cut and sixty-four positions along it, with the shape profile in a fourth channel and in register with them. Asking whether two such strips continue into one another is a texture and edge-continuation question, which is what convolutions are for. And the kernels must be the *same* for both sides, since "does A continue into B" cannot depend on which side was named A - that is exactly a Siamese network.

Input	two (4, 64, 8) tensors: RGB at 8 depths + shape profile. The second is reversed along its length, because mating sides are traversed in opposite directions
Encoder	three convolutional blocks (3x3 conv, BatchNorm, ReLU, twice, then max-pool along the side only), widening 4 -> 32 -> 64 -> 128; global average pool; linear to a 128-d embedding. Weights shared between the two inputs
Head	embeddings combined as <code>[</code>
Output	<code>sigma(logit)</code> = probability the two sides are neighbours
Parameters	345,473 (1.382 MB)

Pooling is applied along the length of the side only. The depth axis is just eight samples deep and each one means something different (distance from the cut), so pooling it away early would destroy the very gradient information that tells the model whether the picture is continuing.

3.2 Model 2 - Graph neural network

Why a different model, not a re-tuned one. The Siamese network judges a pair *in isolation*: two strips in, one probability out, with no knowledge of the rest of the puzzle. That is precisely its weakness, because a jigsaw is a

competition - a side does not need a partner that looks plausible, it needs the partner that looks *more plausible than every other candidate*, and whose own best choice is this side in return.

The graph model is built on that observation:

- **the unit of inference is the whole puzzle**, not the pair. Every side of every piece is a node; every admissible pairing is a candidate edge; one forward pass scores all of them together.
- **the representation is relational, not convolutional**. Nodes carry a flat 45-number descriptor (pooled colour, the shape profile, the side type one-hot, the length and amplitude). There is no convolution anywhere in the model. What refines a node is **message passing** - each side sees its rival candidates and updates its own embedding accordingly, so the score it gives a partner is a contextual judgement rather than an absolute one.
- **sides of the same piece are linked**, so a side also knows what the rest of its own piece looks like - information the Siamese model never receives.

Input	one graph per puzzle: 4N nodes with 45-d features; edges are admissible inter-piece candidates (capped at 24 per side, shortlisted by the classical cost) plus the intra-piece links
Layers	3 × GraphSAGE convolution (mean aggregation), hidden width 96, LayerNorm + ReLU between
Edge head	a candidate (a, b) scored from $[h_a + h_b,$
Output	$\text{sigma}(\text{logit})$ per candidate edge
Parameters	97,441 (0.39 MB) - 3.5x fewer than the Siamese

3.3 The four outputs the brief requires

For any candidate pair, each model reports: **whether they are neighbours** ($\text{sigma}(\text{logit}) \geq 0.5$), a **numerical compatibility score** (that probability), **which sides matched** (the pair of side indices), and the **relative orientation** the pairing implies.

The last is derived, not predicted, and deliberately so - it is determined exactly by which sides matched. With each piece's rotation written as the index of the side facing North, side a facing direction d means $a = \text{rot_a} + d$, and its partner must face the opposite direction, $b = \text{rot_b} + d + 2$. Eliminating d:

$$\text{rot_b} - \text{rot_a} = b - a + 2 \pmod{4}$$

Predicting this with a network instead would add a way to be wrong about something that is already certain. A test asserts the identity holds for all 64 combinations of the two rotations and the direction.

4. Task 3 - Training

Both models are trained under an identical protocol; only the model and the representation it needs differ.

	Siamese CNN	Graph NN
Loss	BCE-with-logits, positive-weighted by the class ratio	same
Optimiser	Adam	Adam
Learning rate	1e-3, halved after 3 epochs without validation improvement	same
Weight decay	1e-4	1e-4
Batch	256 pairs	one puzzle graph (the natural unit)
Epochs	30, early stopping patience 8	same
Augmentation	illumination, contrast, colour, noise	same features, same augmentation
Model selection	best validation AUC	same

Why AUC and not accuracy. The classes are deliberately unbalanced 1:6, so accuracy is dominated by the negatives; and what the assembler actually consumes is a *ranking* of candidate partners rather than a thresholded decision. AUC measures exactly the ranking quality, and is what model selection and early stopping key on. It is computed from rank statistics (the Mann-Whitney form) and is unit-tested against scikit-learn.

Positive weighting is set to the actual negative/positive ratio each epoch, so the loss does not simply learn to answer "not neighbours".

4.1 Training results

	Siamese CNN	Graph NN
Best validation AUC	0.9969 (epoch 16)	0.9909 (epoch 30)
Training AUC at that epoch	0.9915	0.9912
Final training loss	0.1125	0.2053
Final validation loss	0.0659	0.1185
Epochs run	24	30
Wall-clock training	918 s	20 s
Diagnosis	healthy (val AUC 0.997, train-val gap -0.005)	healthy (val AUC 0.991, train-val gap 0.000)

Over- or under-fitting. Neither. Both train/validation AUC gaps at the selected epoch are ~ 0.00 , and both validation losses fall throughout. Note that validation AUC sits *slightly above* training AUC for much of the run, which looks wrong until one remembers that the training batches are augmented and half their negatives are mined from the hardest available candidates, while validation is neither - the training set is deliberately the harder of the two.

The training-cost gap is the first real difference between the models. The Siamese needs 45x longer to train (918 s versus 20 s) for 3.5x the parameters, because it convolves over a 2,048-value tensor per side while the graph model consumes a 45-number descriptor and scores a whole puzzle in one pass.

5. Task 4 - Puzzle assembly

Each model's scores are converted to costs and handed to the **same** assemble routine used in Milestone 1 - with its border constraints, its best-buddy-first tie-breaking, its three-stage dead-end relaxation and its guarantee to return the best arrangement obtained. Each piece is used exactly once and border constraints are respected because those are properties of the assembler, not of the matcher.

The conversion from a probability to a cost is

$$\text{cost} = -\log(p + \text{epsilon})$$

which is the negative log-likelihood of the pairing. It is chosen over $1 - p$ for three reasons: it is monotonically decreasing in p , so the ordering the model intended is preserved exactly; it is **additive** across the seams of an arrangement, so the total the assembler minimises is the negative log-likelihood of the whole arrangement, which is the quantity that ought to be minimised; and it diverges as $p \rightarrow 0$, which keeps hopeless pairings out of the search entirely.

Admissibility - a flat never forms an interior seam, a tab must meet a blank - is applied on top of the model's opinion exactly as for the classical table, so all three methods search the same space.

6. Task 5 - Performance evaluation

All three methods are evaluated on the **same unseen test puzzles**, which neither model saw during training or model selection.

6.1 The standard test split

Measure	Classical	Siamese CNN	Graph NN
Matching top-1	0.932	0.932	0.885
Matching AUC	0.951	0.988	0.975
Best-buddy precision	1.000	0.975	0.953
Neighbour accuracy	1.000	1.000	0.967
Position accuracy	1.000	1.000	0.926
Orientation accuracy	0.985	0.985	0.902
Position and orientation	0.985	0.985	0.902
Complete reconstructions	9/9	9/9	8/9
Reconstruction quality	0.572	0.961	0.874
Parameters	0	345,473	97,441
Model size	0 MB	1.382 MB	0.39 MB
Training time	0 s	918 s	20 s
Inference / puzzle	0.018 s	0.085 s	0.015 s

Orientation accuracy is scored against the rotation each piece held in the finished puzzle, allowing one global quarter turn shared by every piece, since an arrangement that comes out sideways is still an arrangement. It is worth reporting separately from position for a reason visible in the row above: position accuracy is 1.000 for two of the three methods while orientation is 0.985. A piece can sit in the correct cell turned the wrong way, and the position measure alone calls that a success. Roughly one piece in seventy is placed correctly and oriented wrongly - a small effect, but one that only a separate measure can see, which is presumably why the brief asks for both.

The ground truth for it is not stored anywhere. It is implied by the labels already present: a piece's rotation is the index of the side facing North, and every true seam pins that down, because the grid says which way the seam runs. `PuzzleSample.true_rotations` derives it, and a test checks the derivation against the generator's own record for every piece of a puzzle.

This table is saturated, and saying so matters more than the numbers in it. Every method reconstructs essentially every test puzzle perfectly, so on the measure that counts - did the puzzle come out right - the comparison cannot separate them. A ceiling is not a result; it means the test was too easy. Two things follow.

First, the measures *underneath* the ceiling do separate them, and in a consistent direction: the Siamese ranks candidates best (AUC 0.988 against the classical 0.951), and the reconstruction-quality score - how much better than chance the seams actually paid are - rises from 0.57 for the classical measure to 0.96 for the Siamese. All three are equally *correct* here; the learned matchers are far more *confident*, which is what buys margin when the input gets harder.

Second, a harder test is needed before any claim about accuracy can be made (§6.3, §6.4).

6.2 How performance changes with puzzle size

AUC / neighbour accuracy, and matching time per puzzle in seconds:

Grid	Classical	Siamese	Graph NN	t class.	t Siam.	t GNN
3x4	0.926 / 1.00	0.980 / 1.00	0.965 / 0.93	0.009	0.054	0.008
4x5	0.972 / 1.00	0.992 / 1.00	0.976 / 1.00	0.015	0.094	0.015
5x7	0.969 / 1.00	0.998 / 1.00	0.991 / 1.00	0.038	0.162	0.025

Two things move in opposite directions as the puzzle grows.

Accuracy improves. Every method's AUC rises with piece count - the Siamese from 0.980 at 3x4 to 0.998 at 5x7, the graph model from 0.965 to 0.991. A bigger puzzle brings more constraints as well as more pieces, and the border/flat rules bite harder as the perimeter grows.

Cost grows. Matching time roughly triples from 3x4 to 5x7 for every method, because the number of candidate side pairs goes as the square of the piece count.

The graph model's single imperfect run is on the *smallest* grid, and that is explicable rather than random: its whole mechanism is to judge a side against its rivals, and a 3x4 puzzle offers the least context to do that with. The model that depends on context is the one that suffers when there is least of it.

6.3 Breaking the ceiling: fading the picture out

Milestone 1 established that colour carries most of the discriminative power, and that the dataset's own puzzle defeats the classical measure precisely because its picture is nearly uniform. That failure mode can be reproduced on demand by fading a generated picture towards flat grey - texture = 1 is the ordinary picture, texture = 0 is featureless - which withdraws the photometric signal while leaving everything else alone.

Neighbour accuracy, four 4x5 puzzles per level:

Texture	Classical	Siamese CNN	Graph NN
1.00	1.00	1.00	0.94
0.60	1.00	1.00	1.00
0.35	1.00	1.00	0.96
0.20	1.00	1.00	0.97
0.10	0.84	1.00	0.81

This is the result the standard split could not give. Down to texture 0.20 every method is perfect and the comparison stays saturated. At texture 0.10 the hand-designed formula breaks - 0.84 - while **the Siamese CNN still reconstructs every puzzle perfectly**. The learned matcher is not merely better calibrated; it keeps working after the classical one has run out of signal, and it does so in exactly the regime that matters, because that regime is where the real dataset lives.

The graph model does not share the advantage (0.81 at texture 0.10, and the weakest of the three at full texture too). Its node descriptor pools colour into eight bins along the side, which is enough when the picture is informative but discards the fine variation that is all that remains when it is not. The Siamese, convolving over the full 64x8 strip, still has something to work with. That is a representation difference, not a capacity difference: the graph model is the smaller network but it is also the blunter *input*.

6.4 The real dataset photographs

The final test, and a genuine domain shift: both models were trained on generated puzzles and are here asked about photographs of a real jigsaw. Eight full-scramble photographs, scored against the answer key recovered in Milestone 1 §9.1:

The classical baseline is run here in **its own Milestone 1 configuration** (colour_norm="meanstd", main.DATASET_SOLVER). That matters: Milestone 1 §6 adds that normalisation specifically because the pieces lie under uneven light, and measured it lifting top-1 from 0.193 to 0.251. An earlier version of this evaluation called `build_compatibility` with its defaults and so compared the networks against a classical method with its illumination correction switched off. The corrected figures are below; the earlier ones understated the baseline by 0.05 neighbour accuracy.

Measure	Classical	Siamese CNN	Graph NN
Neighbour accuracy	0.222	0.224	0.157
Position accuracy	0.138	0.092	0.121
Reconstruction quality	0.120	0.555	0.561

None of the three reconstructs the real puzzle, and none of them is meaningfully ahead. Classical and Siamese are separated by 0.002 of neighbour accuracy - under one percent, far inside the spread across the eight images (0.16 to 0.28) - and on *position* accuracy the classical measure is the best of the three. Roughly one adjacency in five is all any method achieves, against about one in twenty by chance.

That is a more useful result than the one this section previously reported. Learning does not help here. Whatever the Siamese gains on the low-texture sweep, where the difficulty is a single controlled variable, it does not carry across to photographs that vary illumination, white balance, perspective and cut geometry all at once.

The reconstruction-quality column must not be read across methods at all. It normalises each arrangement's mean seam cost by the mean of that method's *own* cost table, and the learned tables are $-\log p$ while the classical one is a weighted RMS distance - scoring a single fixed arrangement with all three tables gives 0.53, 0.99 and 0.98 for the same arrangement. The column is a valid relative measure within one method and meaningless between them.

Why the domain shift is severe is not mysterious. The texture sweep (§6.3) changes one thing at a time and the Siamese survives it. The photographs change several at once: a nearly white picture, per-piece illumination and white balance, perspective differences between the centre and the edge of the frame, and a die-cut geometry with a kerf that the generator does not model. Training on generated puzzles cannot cover that, and the honest conclusion is that the ceiling here is the training distribution rather than the model.

6.5 Which method is best

The brief asks for the most accurate and the most computationally efficient method. They are not the same one, and neither answer is unqualified.

Most accurate: the Siamese CNN, on generated puzzles only. It ranks candidates best on the standard split (AUC 0.988 against 0.951) and it is the only method that survives the low-texture condition intact (1.00 where the classical formula falls to 0.84). That is the whole of its advantage: on the real photographs it ties the classical measure on neighbour accuracy (0.224 against 0.222) and loses to it on position accuracy (0.092 against 0.138). It costs 345 k parameters, 1.4 MB and sixteen minutes of training to get there, and the claim rests entirely on generated data.

Most efficient: the graph neural network - and by a wide margin. It reaches AUC 0.975, within 0.013 of the Siamese, using **3.5x fewer parameters** (97 k, 0.39 MB), **47x less training time** (21 s against 968 s) and inference *as fast as the hand-written classical formula* (0.015 s against 0.017 s per puzzle, where the Siamese needs 0.088 s). For a matcher that has to run inside a search loop, that is the difference that would matter in practice. Its weakness is the blunter input representation, not the architecture: it pools colour into eight bins and therefore has least to say exactly when the picture has least to give.

Most economical overall: the classical measure, which needs no training data, no training time and no parameters, matches the learned models wherever the picture is informative, and - once run in its own configuration - matches them on the real photographs too. It should be preferred unless the input is genuinely hard in the one specific way the sweep isolates: a picture whose texture has been withdrawn while everything else stays constant. That boundary sits at around texture 0.15, and the dataset's own photographs are not on the far side of it in a way any of these methods can exploit.

6.6 What was learned

1. A saturated benchmark hides everything.

All three methods reconstruct every standard test puzzle perfectly. Had the evaluation stopped there, the report would have concluded that the models are equivalent, which the texture sweep shows is false.

2. Learning buys robustness on one controlled axis, and nothing else.

Where the picture is informative the hand-designed formula is already at the ceiling. The learned matchers earn their keep only as texture is withdrawn with everything else held constant - and on the real photographs, which vary everything at once, they do not beat the classical measure at all. That is a narrower and better-supported claim than a general preference for neural networks.

3. Representation beat capacity.

The larger model won, but the mechanism was the richer input (a 64x8 strip versus eight pooled colour bins), not the extra parameters. The graph model's contextual message passing gave it nearly the same AUC on a fraction of the budget.

4. A reference-free quality score cannot be read across methods.

The reconstruction-quality figure normalises by each method's own cost distribution, so it says nothing about which arrangement is better: scoring one fixed arrangement with all three tables returns 0.53, 0.99 and 0.98. An earlier draft of this report read the gap as evidence that the learned models were confidently wrong. It was evidence about the normalisation. The score remains useful for ranking arrangements *within* one matcher and must not be used between them.

5. The binding constraint is the training distribution. The Siamese

handles low texture when that is the only change, and fails on photographs that change texture, lighting, perspective and cut geometry at once. The next step for this project is not a bigger network but training data drawn from the photographs themselves - which needs side-level labels the dataset does not currently carry.

7. Reproducing

```
pip install -r requirements.txt           # includes torch and torch-geometric
python main_milestone2.py --scaling      # trains both models and compares
python main_milestone2.py --hard --figures # the hard tests, plus every figure
python tests/run_tests.py ml_models      # the Milestone 2 tests
```

Everything is written to results/milestone2/: dataset.json (split and class balance), training.json (architectures, hyper-parameters, per-epoch curves), comparison.json (the three-way comparison), scaling.json (accuracy against puzzle size), texture_sweep.json and real_photographs.json (the two hard conditions), predicted_matches.json (the four required per-pair outputs) and the two trained checkpoints.

7.1 The reconstructed images

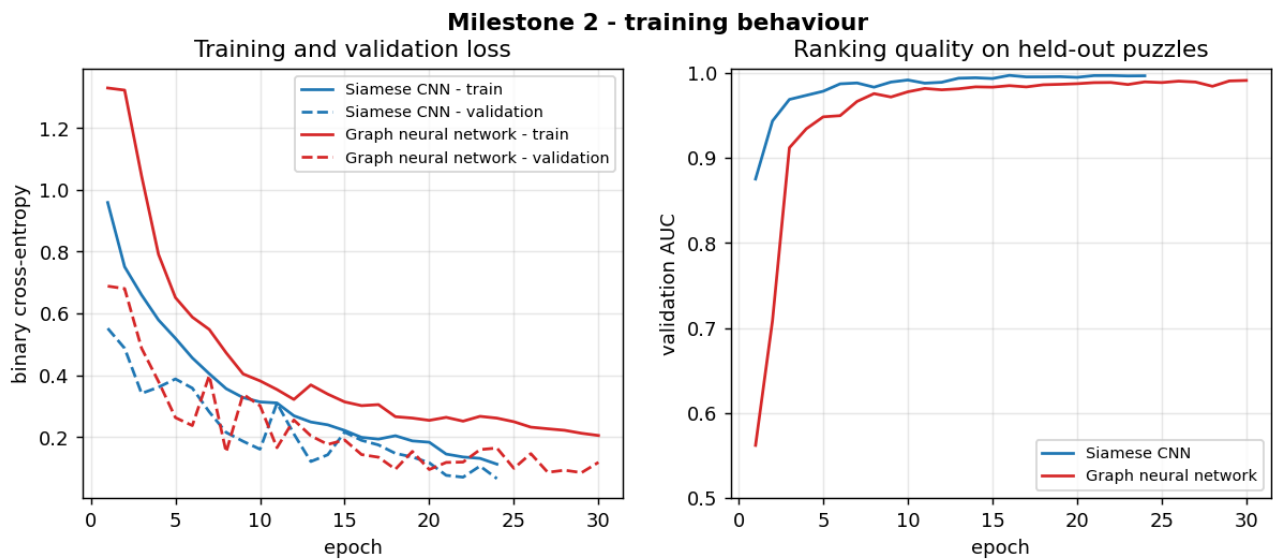
--figures writes results/milestone2/figures/: three charts summarising the tables above - training_curves.png, test_split_comparison.png, texture_sweep.png - and eight reconstructed images, real_0718-10... to real_0718-19..., one per dataset photograph. Each shows the three methods side by side on the same photograph, captioned with its measured neighbour and position accuracy; figures/index.json records the same numbers.

Only the real photographs are drawn. The generated puzzles are what the models are trained and scored on, and §6.1-6.3 report those numbers in full, but a picture of a generated puzzle being solved shows nothing its numbers do not - so the images kept here are of the actual jigsaw.

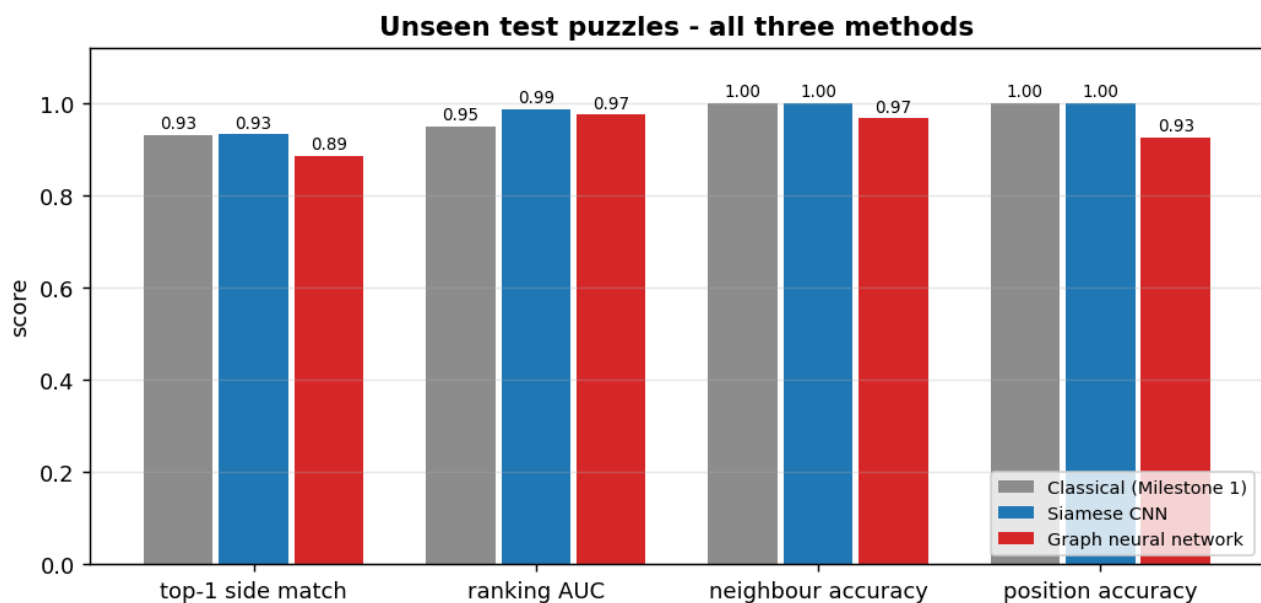
That makes every reconstructed image in this report a failure, which is the point of including them. The captions carry the numbers deliberately: the panels show thirty-five pieces tiled into a clean rectangle with barely a fifth of them in the right cell, and that is exactly the arrangement a viewer would otherwise take for a success.

Appendix - Figures

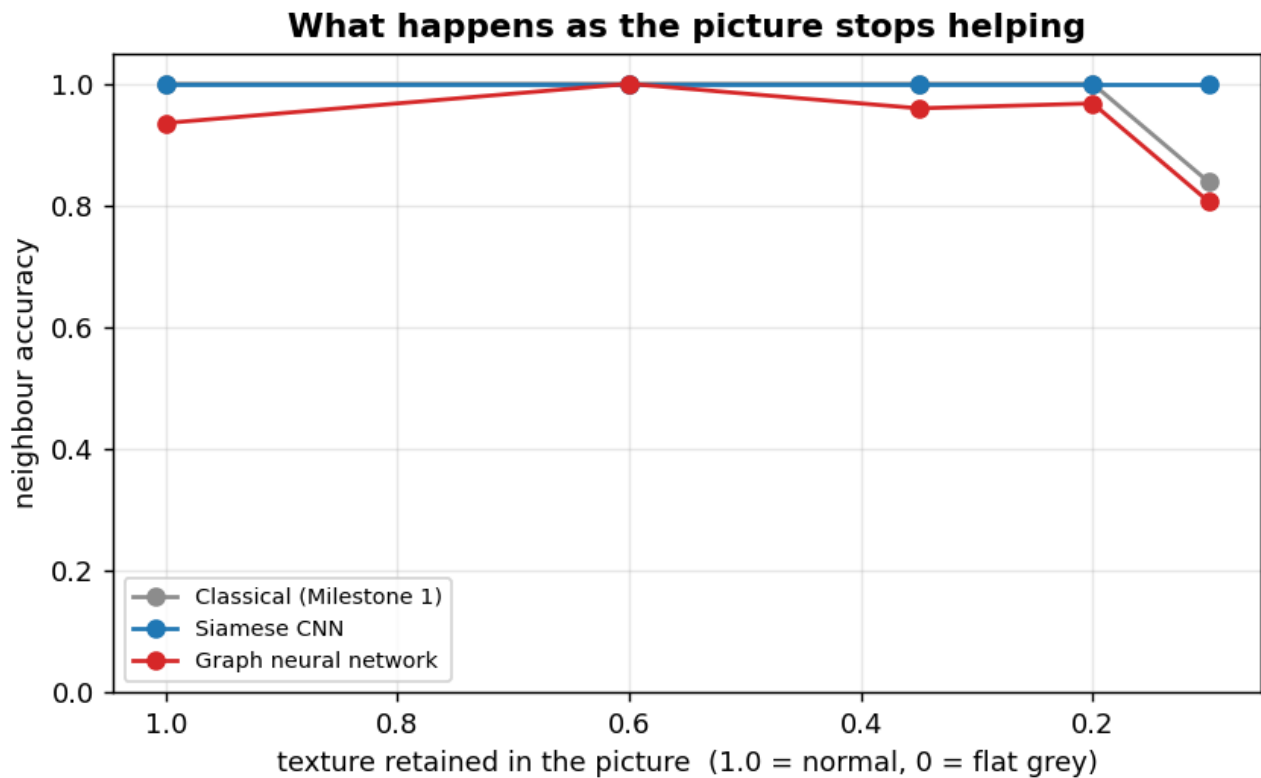
Every figure below is produced by `python main_milestone2.py --hard --figures` and written to `results/`.



`milestone2/figures/training_curves.png` — Task 3 - training behaviour: cross-entropy per epoch and the validation AUC both models were selected on. The train and validation curves stay together, which is why both are diagnosed healthy.

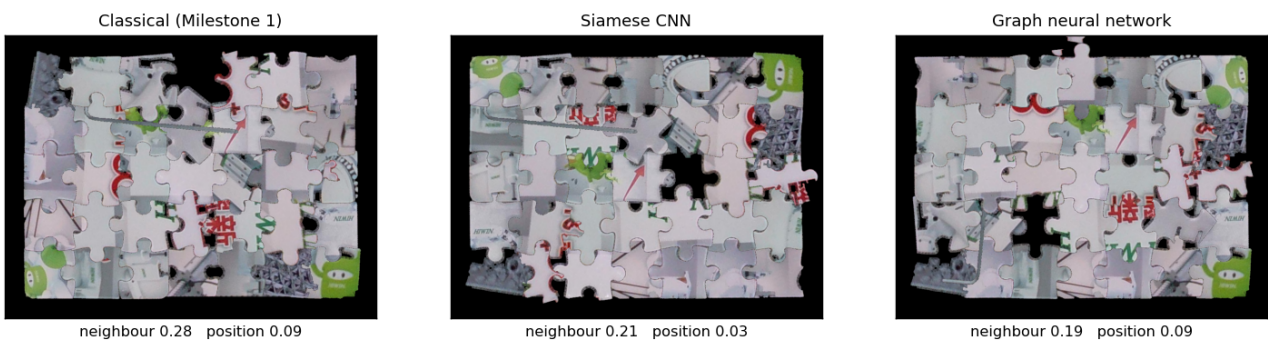


`milestone2/figures/test_split_comparison.png` — Task 5 - the unseen test split. Every method reconstructs every puzzle, so this comparison is saturated and cannot rank them; only the ranking quality (AUC) separates the models here.



milestone2/figures/texture_sweep.png — Task 5 - neighbour accuracy as the picture's texture is faded towards flat grey. The hand-written measure collapses at texture 0.10; the Siamese network does not.

Dataset photograph - 0718-10_Color_png.rf.f269500b6bdb1b871bfc5b9c53814026.jpg



milestone2/figures/real_0718-10_Color_png.rf.f269500b6bdb1b871bf.png — Task 5 - a dataset photograph, reconstructed by all three methods. None of them solves it: the pieces tile into a neat rectangle and hardly any of them are in the right cell.

Dataset photograph - 0718-14_Color_png.rf.7ba095874a51a7e4c9516427b281ed77.jpg

Classical (Milestone 1)



neighbour 0.21 position 0.09

Siamese CNN



neighbour 0.28 position 0.06

Graph neural network

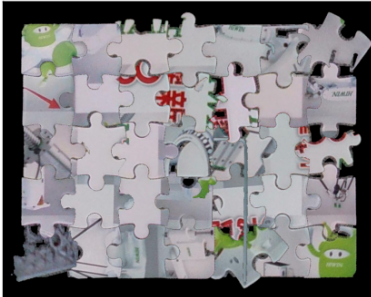


neighbour 0.16 position 0.20

milestone2/figures/real_0718-14_Color_png.rf.7ba095874a51a7e4c95.png — The photograph on which the Siamese network does best (neighbour 0.28 against the classical measure's 0.12) - still far from a solution.

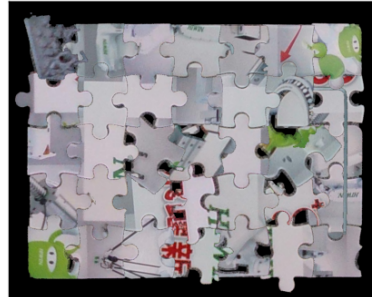
Dataset photograph - 0718-18_Color_png.rf.7ce49fe96d1498ecbab4d952cca0c910.jpg

Classical (Milestone 1)



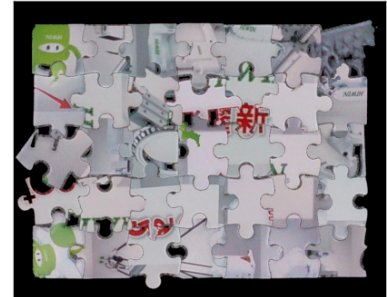
neighbour 0.24 position 0.17

Siamese CNN



neighbour 0.21 position 0.06

Graph neural network



neighbour 0.10 position 0.09

milestone2/figures/real_0718-18_Color_png.rf.7ce49fe96d1498ecbab.png — A third photograph. Across all eight the learned models report much better seam quality while placing no more pieces correctly, which is what being confidently wrong outside the training distribution looks like.